UT04.Act07. Optimizando y documentando El Lobo

Objetivos

- Aplicar técnicas de refactorización orientada a objetos (POO), herramientas de análisis de código (Ruff), documentación técnica (Sphinx) y control de versiones (Git).

PASO 1: CLONACIÓN Y AUDITORÍA INICIAL.

El código que vas a heredar es un "**código espagueti**": todo está en un mismo archivo, mezclando lógicas, incumpliendo las normas de estilo **PEP 8** y usa malas prácticas.

Tu primer trabajo es identificar el desastre.

1. **Crea un repositorio GIT** para trabajar y súbelo a **GITHub**.
2. Crea una RAMA de trabajo aislada para tus cambios denominada **refactorización-poo.**
3. **Audita** el código con Ruff para detectar errores de estilo PEP8

PASO 2: ARQUITECTURA Y REFACTORIZACIÓN

Vamos a dividir juego.py en dos paquetes bien diferenciados: las fichas del juego (personajes) y las reglas del juego (lógica)

1. Crea la estructura de carpetas y archivos (no olvides los __init__.py para que Python los reconozca como paquetes)
2. Identifica y crea las clases necesarias en personajes. Recuerda que puedes aplicar herencia.
3. Refactoriza la lógica del juego.
4. Prueba que **funciona**: Crea un archivo main.py en la raíz, importa tus paquetes y simula una partida (un lobo ataca, amanece, se vota a un sospechoso).
5. Guarda tu progreso en GIT y GitHub con el siguiente mensaje: "*Refactor: Modularización en paquetes lógica y personajes*"
6. Si todo va bien, realiza un **Merge** al Main.

PASO 3: CALIDAD DEL CÓDIGO

Ahora que el código está en su sitio, vamos a limpiarlo usando Ruff.

1. Crea una RAMA de trabajo aislada para tus cambios denominada **limpieza**
2. Aplica el formato automático (arregla espacios, comillas y sangrías)
3. Aplica correcciones automáticas seguras (elimina importaciones sin usar):
4. Realiza arreglos manuales: Vuelve a lanzar **ruff check .** y corrige a mano lo que el linter no puede adivinar:
    a. Cambia las clases a PascalCase (ej. GestorPartida).
    b. Cambia las variables reservadas (list, dict, type) por nombres descriptivos (lobos_vivos, aldeanos_vivos, rol_asignado).
5. Guarda tu progreso en Git y GitHub con el mensaje: *"Style: Corrección de estilo PEP 8 y code smells con Ruff"*
6. Si todo va bien, realiza un **Merge** al Main.

PASO 4: DOCUMENTACIÓN GUIADA CON SPHINX

Vamos a generar un manual técnico en formato Markdown para que se lea nativamente en GitHub.

1. Añade **Docstrings** en el formato que quieras
2. Crea la carpeta docs en el proyecto y **accede** a ella.
3. Inicia Sphinx: **sphinx-quickstart** (Responde si a separar source/build, pon el nombre del proyecto y tu nombre).
4. Configura Sphinx:
   a. Abre docs/source/conf.py y añade esto:
   ```
   # Al principio del archivo:
   import os
   import sys
   sys.path.insert(0, os.path.abspath('../../'))

   # En la sección de extensions:
   extensions = [
       'sphinx.ext.autodoc',
       'sphinx.ext.napoleon',
       'sphinx_markdown_builder'
   ]
   ```
5. Genera los **índices automáticos**: (Vuelve a la raíz del proyecto cd ..)
   Ejecuta: `sphinx-apidoc -o docs/source/ logica/ personajes/`

6. Construye el **Markdown**:
   Ejecuta: `sphinx-build -b markdown docs/source docs/build/markdown`

   Si da error prueba: python3 -m sphinx.cmd.build -b markdown docs/source docs/build/markdown

PASO 5: ENTREGA FINAL

Debes entregar todo el proyecto subido a GitHub.

1. Crea el **README.md**: En la raíz del proyecto, crea un archivo README.md que enlace a tu documentación generada:
   Markdown
   # 🐴 El Juego del Lobo (Refactorizado)
   * [Ver Documentación del Motor](docs/build/markdown/logica.md)
   * [Ver Documentación de los Personajes](docs/build/markdown/personajes.md)
2. Realiza un commit y sube tu código a GitHub. Mensaje *“Docs: Generación de API en Markdown y simulación final"*

¡Enhorabuena! *Transformaste un código insostenible en una arquitectura limpia, profesional, documentada y lista para producción.*